

**IDEA PHASE SUBMISSION**

# **APK THREAT INTELLIGENCE SYSTEM**

Generative AI-Based Automated Analysis and Risk  
Scoring of Fraudulent APKs

**Bank of India · National Hackathon**  
*Technical Design Documentation*

**TEAM – AEGIS AI**

Alisha Kulkarni  
Ayush Shevde  
Shreeyash Indulkar  
Shravni Thakur

## Contents

---

- 1. Executive Summary**
- 2. Problem Statement**
- 3. Solution Overview**
- 4. System Architecture**
- 5. Technology Stack**
- 6. End-to-End Analysis Workflow**
- 7. Analysis Components**
  - 7.1 Feature-Store Pipeline
  - 7.2 The Capability Rulebook
  - 7.3 Feature-Store Score
  - 7.4 Reverse Engineering (Code-Behaviour Analysis)
  - 7.5 Code-Behaviour Score
  - 7.6 Fusion Engine
  - 7.7 MITRE ATT&CK Mobile Mapping
  - 7.8 Social-Engineering Detector
  - 7.9 Intent-Spoofing (Impersonation) Detector
  - 7.10 APK DNA Fingerprinting
  - 7.11 Campaign Store
  - 7.12 Network-Graph Intelligence
  - 7.13 Multi-Label Malware-Type Classifier
  - 7.14 Report Generation
- 8. Leveraging Generative AI**
- 9. Analyst Console**
- 10. Scalability**
- 11. Future Scope**
- 12. Prototype**
- 13. Conclusion**

# 1. Executive Summary

---

Fraudsters increasingly distribute malicious Android applications through WhatsApp, SMS, email, and phishing links to steal banking credentials, harvest one-time passwords, and authorise unauthorised transactions. Manual triage of these APKs is slow, requires scarce specialist skill, and rarely scales to the volume of samples a bank encounters. **The APK Threat Intelligence System addresses this gap with an automated, explainable, Generative-AI-assisted analysis platform that ingests a suspicious APK and returns a defensible risk verdict, a structured evidence trail, and an analyst-ready incident report.**

The defining principle of the system is a strict separation between deterministic scoring and generative reasoning. A rule-based engine produces an auditable risk score that is fully reproducible and traceable to concrete signals, while a locally hosted large language model interprets decompiled code, recognises social-engineering patterns, and authors the final narrative. The two are deliberately never blended in a way that would compromise auditability — a requirement of any system that supports financial-fraud decisioning.

Around this core, the platform layers a family of complementary detectors: structural malware-family fingerprinting, coordinated-campaign clustering, banking-app impersonation detection, social-engineering analysis of the application's own interface text, MITRE ATT&CK technique mapping, and a multi-label machine-learning classifier. Every signal is surfaced to the analyst through a dedicated console, and the entire evidence set is consolidated by the language model into a single human-readable report.

## 1.1 Document Purpose and Scope

This document is the technical design companion to the idea-phase presentation. It describes the proposed system end to end — its architecture, the role of every analysis component, where and how Generative AI is applied, the analyst-facing console, and the path from the current proof-of-concept to a production deployment. It is written for a mixed audience of technical evaluators and banking and compliance stakeholders.

# 2. Problem Statement

---

Mobile banking fraud driven by malicious APKs has become one of the most damaging vectors in the Indian financial ecosystem. The attack pattern is consistent and well understood:

1. **Distribution.** A fraudster circulates a malicious APK through WhatsApp, SMS, email, or a phishing page, frequently disguised as a bank app, a KYC-update utility, a reward application, or a government service.
2. **Social engineering.** The victim is pressured into installing the app and granting permissions through fabricated urgency — a fake account-suspension notice, a counterfeit RBI or NPCI message, or a fraudulent reward.

3. **Credential and OTP theft.** Once installed, the app abuses accessibility services, screen overlays, and SMS interception to capture credentials and one-time passwords, enabling unauthorised transactions.

Defending against this at scale is constrained by the realities of manual analysis. Reverse engineering an APK to determine what it truly does is time-consuming, depends on a small pool of skilled analysts, and does not keep pace with the rate at which new and repackaged samples appear. Banks need a way to triage suspicious applications quickly, consistently, and with evidence strong enough to support action.

#### **Problem statement — as posed**

Develop a Generative-AI-powered malware analysis system capable of automatically analysing suspicious APK files and identifying malicious behaviour. The system should leverage GenAI for reverse engineering, malware-pattern recognition, automated code interpretation, and intelligent threat summarisation, alongside static and dynamic analysis of permissions, APIs, embedded code, runtime activities, and network communications. Using AI-driven insights, the solution should detect malware patterns, classify threat severity, generate risk scores, and produce detailed investigation reports with actionable recommendations — enabling faster identification of fraudulent applications and supporting proactive fraud prevention for banks.

## **3. Solution Overview**

The system accepts a suspicious APK and runs it through a multi-stage analysis pipeline, producing a single risk verdict supported by a transparent evidence trail. The design is organised around four ideas.

### **3.1 Deterministic scoring and Generative AI are kept separate**

Two independent risk scores are computed for every APK. The **Feature-Store Score** is produced by a deterministic, rule-based engine and reflects what the application *declares* — its permissions, manifest attributes, certificate, and observed network and runtime characteristics. The **Code-Behaviour Score** is produced by a Generative-AI reverse-engineering stage and reflects what the application's code *actually does*. The deterministic score is authoritative and fully auditable; the AI-derived score is advisory. Keeping them separate means the AI never silently alters a number a bank must be able to justify.

### **3.2 The two scores are compared, not averaged**

A deterministic fusion stage reasons over the two scores together. The most valuable outcome it surfaces is divergence: an application that declares almost nothing suspicious yet whose code demonstrably performs dangerous actions. This is the signature of obfuscated or dynamically loaded malware — precisely the case a single lens would miss, and the case the two-score design exists to catch.

### **3.3 A shared rulebook makes the two lenses comparable**

Both scores are evaluated against the same predefined catalogue of named threat capabilities. Because the feature side and the code side measure the same set of capabilities — only through different means

— any disagreement between them is analytically meaningful rather than an artefact of two unrelated measurement schemes. The same rulebook also carries the MITRE ATT&CK mapping, so every confirmed capability is automatically expressed in an internationally recognised framework.

### 3.4 Complementary detectors add breadth

Beyond the two-score core, the platform runs a set of standalone detectors that each answer a distinct question — does this app impersonate a real bank, does it resemble a known malware family, is it part of a coordinated campaign, does its interface attempt to manipulate the user — and surfaces each as its own evidence panel. A final Generative-AI stage consolidates everything into one report.

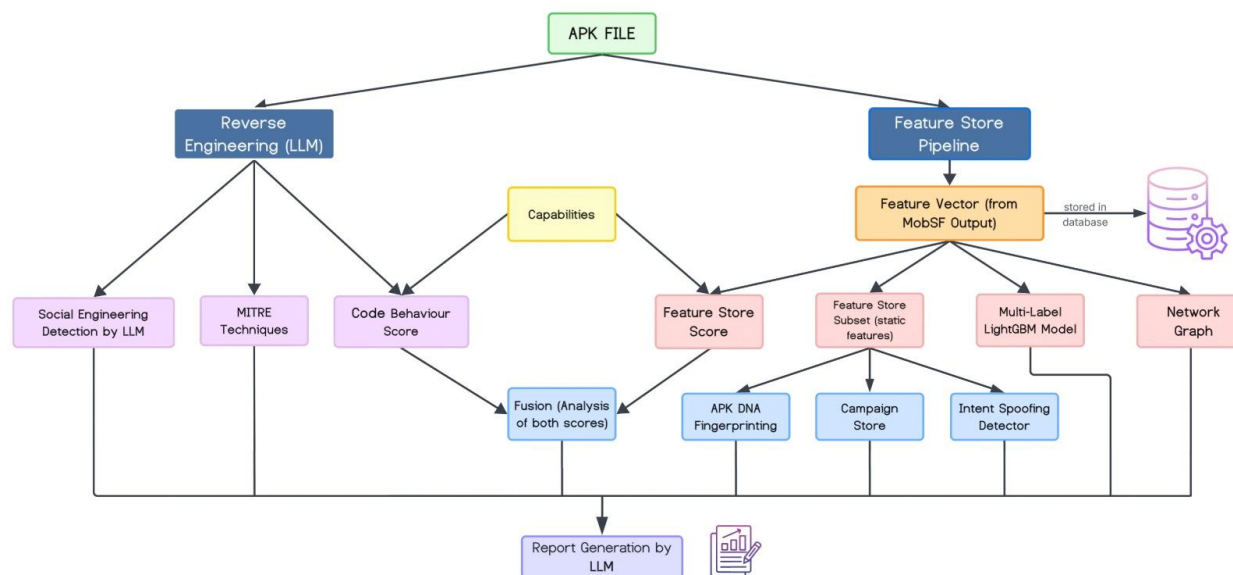
### 3.5 Headline capabilities

- **Reproducible static and dynamic feature extraction** from each APK into a structured feature set.
- **Generative-AI reverse engineering** of decompiled code to confirm what an application actually does.
- **Dual risk scoring with deterministic fusion** into a single, defensible verdict and recommended action.
- **MITRE ATT&CK Mobile mapping** of every confirmed capability.
- **Social-engineering detection** over the application's own interface text, tuned for the Indian banking context.
- **Banking-app impersonation detection** through certificate and package verification against a whitelist of genuine apps.
- **Structural malware-family fingerprinting and campaign clustering** to relate a sample to known families and to other analysed samples.
- **Multi-label malware-type classification** as an experimental cross-check.
- **Network-graph intelligence** linking applications, infrastructure, certificates, and domains.
- **Generative-AI report generation** consolidating all evidence into an analyst-ready incident report.

## 4. System Architecture

---

The platform is organised into four logical tiers: an input layer that receives the APK, an ingestion and storage layer that holds extracted features and fingerprints, an analysis and reasoning engine that performs all scoring and detection, and a reporting layer that produces the final consolidated output. A single backend service orchestrates the flow and streams live progress to the analyst console.



## 4.1 Logical tiers

- **Input layer** — Receives the suspicious APK uploaded through the console. Optional metadata and alert context can accompany the sample.
- **Ingestion and storage layer** — Persists each APK's extracted feature set and its structural fingerprint so that downstream components share a single source of truth and so that history and cross-sample comparison are possible. The proof-of-concept uses a local database; the production design moves this to a scalable store and a vector index (see Section 9).
- **Analysis and reasoning engine** — The heart of the system. It runs static and dynamic analysis, the two risk scores and their fusion, and every standalone detector. It hosts the local language model used for reverse engineering, social-engineering analysis, and report writing.
- **Reporting layer** — Consolidates the outputs of every component into a structured incident report written by the language model, presented alongside each component's own evidence panel.

## 4.2 Orchestration and live progress

A full analysis is a multi-minute operation — static analysis, then best-effort dynamic analysis, then several language-model passes. The backend therefore runs each analysis as a background job and streams stage-by-stage progress events to the console in real time, so the analyst watches the pipeline advance rather than waiting on an opaque request. When the job completes, the console retrieves the full result for display. The same persisted results power the history view, allowing any past analysis to be reopened.

## 4.3 Local-first and privacy-preserving

All analysis — including the language-model inference used for reverse engineering and reporting — runs locally. Suspicious samples, decompiled code, and their extracted evidence never leave the environment for a third-party cloud service. For a bank handling potentially sensitive artefacts, this independence from external AI services is a deliberate and important property.

## 5. Technology Stack

The platform composes a small set of proven open-source tools, each chosen for a specific role in the pipeline. The table below summarises the analysis and reasoning components; the subsequent table covers the supporting infrastructure described in the production design.

### 5.1 Analysis and reasoning components

| Component                         | Role in the system                                                                                                                                                                                                                                      |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MobSF (Mobile Security Framework) | Performs static and dynamic analysis of the uploaded APK, extracting permissions, API usage, certificate properties, network indicators, runtime behaviour, and anti-analysis signals. Its findings feed every downstream scoring and detection module. |
| JADX                              | Reverse-engineers the APK into readable Java source code. The decompiled output is what the language model interprets to determine the application's true behaviour, and its resource files supply the interface text analysed for social engineering.  |
| Ollama (local LLM runtime)        | Hosts the language model locally, enabling code interpretation, social-engineering pattern recognition, and report generation entirely on-premises — ensuring privacy and independence from external cloud AI.                                          |
| LightGBM                          | Powers the prototype multi-label malware-type classifier, predicting threat categories from the extracted feature set as an experimental cross-check.                                                                                                   |

### 5.2 Supporting infrastructure (production design)

| Component    | Role in the production deployment                                                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL   | Centralised feature store for static, dynamic, and behavioural indicators, supporting scalable storage, querying, and management across large sample volumes.           |
| Vector store | Stores malware fingerprints as vector embeddings, enabling efficient similarity search, family identification, campaign clustering, and large-scale threat correlation. |
| Docker       | Containerises analysis services and AI components for reproducible deployment, environment consistency, and portability across development and production.              |
| Apache Kafka | Enables distributed, asynchronous APK processing so that large volumes of samples can be analysed concurrently in a scalable pipeline.                                  |

## 6. End-to-End Analysis Workflow

When an analyst submits an APK, the backend executes the following sequence. Each stage emits a progress event, and the deterministic stages never depend on the availability of the language model — if AI inference is unavailable, the system degrades gracefully and still returns a deterministic verdict.

1. **Static analysis:** MobSF inspects the APK without executing it, extracting permissions, manifest attributes, certificate properties, API usage, network endpoints, code patterns, and obfuscation indicators.
2. **Dynamic analysis:** The application is executed in an instrumented Android emulator and its runtime behaviour is recorded. This stage is best-effort: if it cannot run, the pipeline continues on static features alone.
3. **Feature extraction and persistence:** Both reports are normalised and distilled into a single structured feature set, which is stored and made available to every downstream component.
4. **Impersonation check:** The application's package name and signing certificate are compared against a whitelist of genuine bank apps to detect forgery.
5. **Feature-Store Score:** The deterministic engine scores the feature set against the capability rulebook, producing an auditable risk score and band.
6. **Malware-type classification:** The prototype classifier predicts threat categories from the feature set as an experimental cross-check.
7. **DNA fingerprinting:** A structural fingerprint is built and compared against a reference database of known malware families; the sample's own fingerprint is stored for future comparison.
8. **Campaign clustering:** The sample's relationships to previously analysed APKs are computed to reveal coordinated campaigns.
9. **Reverse engineering:** JADX decompiles the APK; the language model interprets the highest-priority code slices to confirm behaviours, producing the Code-Behaviour Score and the MITRE map.
10. **Social-engineering detection:** The application's interface text is analysed for fraud and manipulation patterns.
11. **Score analysis (fusion):** The two scores are compared to produce the final quadrant verdict and recommended action.
12. **Report generation:** The language model consolidates every component's output into a structured incident report.

### Graceful degradation

Every Generative-AI stage degrades cleanly. If decompilation or the language model is unavailable, the affected component returns a structured result indicating so, and the pipeline falls back to the



deterministic Feature-Store Score alone. No single component failure can crash an analysis — the deterministic verdict is always produced.

## 7. Analysis Components

This section describes each component of the analysis engine in turn — what question it answers, how it works, and how its output is used. The components fall into three groups: the feature foundation, the two-score core and its fusion, and the standalone detectors.

### 7.1 Feature-Store Pipeline

The feature-store pipeline is the entry point of the entire system. It accepts an APK, orchestrates its analysis through MobSF, and distils the resulting reports into a single structured feature set — the common input format that every downstream component consumes. Nothing else runs until this stage completes.

#### How it works

1. **Static analysis.** The APK is submitted to MobSF, which performs a comprehensive static analysis covering permissions, manifest attributes, certificate properties, API usage, network endpoints, code patterns, and obfuscation indicators.
2. **Dynamic analysis.** The APK is instrumented and executed inside an Android emulator connected to MobSF, and its runtime behaviour is recorded. The stage is best-effort and never blocks the pipeline.
3. **Feature extraction.** Both reports are normalised — tolerating differences between MobSF versions — and distilled into a structured feature set spanning permission flags, API-usage characteristics, network indicators, certificate properties, manifest flags, obfuscation signals, and dynamic runtime behaviour.
4. **Persistence.** The feature set is stored, keyed to the APK, so that all downstream components share the same data without re-running MobSF.

#### Feature groups

The extracted feature set is organised into intuitive groups, illustrated below:

| Group            | Representative signals                                                                            |
|------------------|---------------------------------------------------------------------------------------------------|
| Permissions      | SMS, accessibility, device administrator, screen overlay, camera, microphone, contacts, call log. |
| API usage        | Reflection, cryptography, inter-process communication, file access, network calls.                |
| Network (static) | Domain counts, cleartext-traffic ratio, sanctioned or known-bad domains.                          |

| Group             | Representative signals                                                                   |
|-------------------|------------------------------------------------------------------------------------------|
| Network (dynamic) | Domains contacted at runtime and how they differ from those declared statically.         |
| Certificate       | Debug-signed, self-signed, and the signing-certificate fingerprint.                      |
| Manifest          | Debuggable flag, backup allowance, exported components.                                  |
| Obfuscation       | Compiler and packer indicators, suspicious naming patterns.                              |
| Dynamic runtime   | Clipboard access, local-database access, and whether the sandbox run completed.          |
| Derived           | Composite signals such as the SMS-interception combination and anti-emulation detection. |

**Sandbox-evasion awareness.** If an application carries anti-analysis indicators yet produces no observable behaviour during the dynamic run, the combination is explicitly flagged as suspected sandbox evasion — the app likely detected it was being watched and stayed quiet.

## 7.2 The Capability Rulebook

The capability rulebook is the analytical foundation of the scoring architecture. It is a fixed, predefined catalogue of named threat capabilities, each representing a discrete malicious behaviour characteristic of Android banking malware — for example OTP interception, accessibility abuse, screen overlay, device-administrator acquisition, and dynamic code loading.

Both scoring components evaluate against this same rulebook. This is what makes their outputs directly comparable: the feature side and the code side measure the *same* set of capabilities through two different lenses, so any divergence between them is meaningful rather than an artefact of two unrelated schemes.

### What each capability defines

| Field             | Meaning                                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------|
| Weight            | The risk contribution the capability makes to whichever score it fires in.                              |
| Feature condition | The declared signals — from the feature set — that indicate the capability is present.                  |
| Behaviour tags    | The confirmed code behaviours — emitted by the reverse-engineering stage — that confirm the capability. |
| MITRE technique   | The corresponding MITRE ATT&CK Mobile technique, carried automatically with the capability.             |

**Capabilities the features cannot see.** A small number of capabilities — notably dynamic code loading and shell-command execution at runtime — cannot be detected from declared permissions or manifest flags

at all, because there is no such flag to declare. These surface *only* through code analysis, and they represent exactly the class of hidden threat the two-score architecture exists to expose.

**Weak signals are bounded.** Some signals — use of reflection, a self-signed certificate, a debuggable build — are only mildly suspicious and are routinely tripped by a legitimate development build. The combined contribution of these weak signals is bounded so that an ordinary debug build cannot accumulate a falsely elevated score on weak signals alone.

### 7.3 Feature-Store Score

The Feature-Store Score is the deterministic, fully auditable side of the scoring system. It evaluates the feature set against the capability rulebook: every capability whose declared condition is satisfied contributes its predefined weight, the total is bounded and mapped to a risk band — **Low, Medium, High, or Critical** — and the result answers the question, “what does this application *declare* it can do?”

Because the score uses no machine learning and no randomness, the same APK always produces the same score, and every point is traceable to a specific signal. A bank's fraud team can read the output and reconstruct exactly why an application scored what it did — a property essential for any decision that may need to be defended. The score is accompanied by an evidence trail that cites, for each fired capability, the concrete signals that triggered it.

### 7.4 Reverse Engineering (Code-Behaviour Analysis)

Reverse engineering is the Generative-AI side of the scoring system. Where the Feature-Store Score asks what an application *declares*, this stage asks what its code *actually does* — independently of declared permissions or manifest flags.

#### How it works

1. **Decompile.** JADX converts the APK back into readable Java source code.
2. **Triage.** Rather than reading the entire codebase, the system uses the MobSF report as a map to identify the highest-priority suspicious methods, and analyses only those.
3. **Interpret.** Each prioritised code slice is sent to the local language model, which is constrained to describe what it sees using only the canonical behaviour tags defined in the capability rulebook. The model cannot invent its own labels — its output plugs directly into the shared scoring system.
4. **Score.** The confirmed behaviour tags are evaluated against the capability rulebook to produce the Code-Behaviour Score and band.

Because a language model is involved, the exact score can vary slightly between runs. The system therefore treats the score as **advisory** and uses only its band for downstream decisions. If decompilation or the model is unavailable, the stage returns a structured “unavailable” result and the pipeline falls back to the deterministic score alone.

## 7.5 Code-Behaviour Score

The Code-Behaviour Score is the value produced by the reverse-engineering stage. It evaluates risk based on behaviours directly observed and confirmed in the decompiled source, and it captures three things the Feature-Store Score structurally cannot:

- Capabilities that require no declared permission — such as dynamic code loading or shell-command execution at runtime.
- Cases where a permission is declared but the code never actually exercises it.
- Cases where dangerous behaviour is implemented but deliberately obfuscated to evade static detection.

As with the reverse-engineering stage that produces it, only the band of this score is used for decisions, reflecting its advisory, AI-derived nature.

## 7.6 Fusion Engine

The fusion engine is the terminal scoring step. It compares the two scores against each other — by band, not by exact value — and emits a single verdict and a recommended action. Reasoning by band reflects the advisory nature of the Code-Behaviour side. The outcome is expressed as one of four quadrants, plus two special cases.

| Situation                       | Interpretation                                                                                                                                                                                 | Action              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| Both high                       | Both lenses agree the application is dangerous: it declares dangerous capabilities and the code was confirmed exercising them. A confirmed threat.                                             | Block               |
| Code high, features low         | The application declares little but its code demonstrably does something dangerous — the obfuscated or dynamically loaded payload case. The most important outcome the design exists to catch. | Block               |
| Features high, code low         | A dangerous capability is declared but reverse engineering did not confirm it being exercised. Likely over-permissioned rather than actively malicious.                                        | Monitor             |
| Both low                        | Neither lens found anything significant. Benign.                                                                                                                                               | Clear               |
| Near a band boundary            | One or both scores sit too close to the Medium/High cut to decide automatically.                                                                                                               | Escalate for review |
| Reverse engineering unavailable | Falls back to the deterministic Feature-Store Score alone and acts on its band.                                                                                                                | Per FS band         |

**Capability gap.** Alongside the verdict, fusion computes a capability gap that classifies every capability as declared-but-not-confirmed (over-permissioning), confirmed-but-not-declared (code doing something no feature flagged — the most interesting case), or agreed by both lenses (high confidence). This gap is the explicit, human-readable expression of where the two lenses agree and disagree.

## 7.7 MITRE ATT&CK Mobile Mapping

Every capability in the rulebook carries a corresponding MITRE ATT&CK Mobile technique. When a capability fires — from either the feature side or the code side — its technique is surfaced automatically; no separate mapping step is required. The confirmed techniques are grouped by ATT&CK tactic to produce a structured coverage map for every analysis.

This grounds the system's findings in an internationally recognised threat framework. It lets a bank's fraud team correlate detections with known adversary techniques, share findings with external security teams and regulators in a standardised format, and prioritise response by tactic. Representative techniques covered include:

| Technique                        | Tactic               | Capability                                                            |
|----------------------------------|----------------------|-----------------------------------------------------------------------|
| T1417 Input Capture              | Credential Access    | OTP and credential theft via SMS interception and accessibility abuse |
| T1516 Input Injection            | Impact               | Accessibility-service exploitation                                    |
| T1626 Abuse Elevation Control    | Privilege Escalation | Device-administrator acquisition                                      |
| T1407 Download New Code          | Defense Evasion      | Dynamic code loading at runtime                                       |
| T1636 Protected User Data        | Collection           | Unauthorised SMS, contact, and call-log access                        |
| T1437 Application Layer Protocol | Command and Control  | Communication with command-and-control infrastructure                 |

## 7.8 Social-Engineering Detector

Banking malware does not rely on technical exploitation alone — it manipulates users psychologically, through fake RBI or NPCI notices, fabricated account-suspension urgency, counterfeit OTP screens, and instructions designed to elicit credential disclosure. The social-engineering detector analyses the application's own interface text to identify these fraud patterns before a customer would be exposed to them. It runs alongside the two-score core as an independent signal and contributes its own evidence panel.

### How it works

1. **Collect.** Interface strings are gathered from the most reliable sources first — the decompiled resource files, then MobSF-extracted strings, then any in-app URLs.
2. **Filter.** Strings are scored for banking and social-engineering relevance, and framework noise, layout labels, and other non-interface text are discarded.
3. **Analyse.** The filtered strings are sent to the local language model with a prompt tuned for the Indian banking context — RBI and NPCI impersonation, UPI fraud, OTP phishing, and account-suspension threats.

4. **Validate.** Any finding that does not correspond to text actually present in the application is discarded, so the model cannot report manipulation that was not there.

The detector recognises the recurring techniques of Indian banking fraud: impersonation of trusted authorities, fabricated urgency, counterfeit credential-entry screens, deceptive interfaces, and phishing destinations.

## 7.9 Intent-Spoofing (Impersonation) Detector

This detector identifies whether an unknown APK is fraudulently impersonating a genuine Indian banking application. It maintains a whitelist of real bank apps, each recorded with its package name and its signing-certificate fingerprint — a unique cryptographic identity only the genuine publisher can produce.

For an unknown APK, two conditions are checked together: whether its package name is confusably similar to a genuine bank's, using fuzzy matching that catches deliberate variations; and whether its signing certificate differs from that bank's. When both hold — a look-alike identity backed by the wrong certificate — the result is treated as a definitive forgery. A fraudster can copy a name and an icon, but cannot forge a signing certificate, which is what makes this signal near-irrefutable. The result is surfaced as its own panel and does not feed the fusion verdict.

## 7.10 APK DNA Fingerprinting

DNA fingerprinting asks whether an unknown APK resembles a known malware family — not by what it does, but by what it looks like structurally. Two complementary similarity layers run in tandem, each catching what the other misses.

- **Feature-vector similarity** — The static portion of the feature set — the permission constellation, API-usage patterns, certificate properties, and network characteristics — is compared mathematically against a reference database of known malware families. Structural resemblance that survives renaming and obfuscation is captured here and reported as a similarity percentage.
- **Code fuzzy-hashing** — A locality-sensitive fuzzy hash is computed over the application's compiled code. Because similar code produces similar hashes, repackaged clones — where an attacker took a known sample, swapped the icon and name, and re-signed it — are identified through byte-level proximity even when their declared features differ.

Each newly analysed APK's fingerprint is stored, so every analysis enriches the reference pool and feeds the campaign clustering described next.

## 7.11 Campaign Store

Where DNA fingerprinting asks which known family a sample resembles, the campaign store asks which other APKs the system has itself analysed are connected to this one — revealing coordinated fraud operations that no single analysis would expose. Two analysed APKs are linked when any one of the following holds: they share a signing certificate (the same actor signed both), they share command-and-control infrastructure (a common malicious domain), they are byte-level code clones (repackaged from the same base), or they are structurally near-identical. A group of two or more linked APKs forms a

campaign. The store grows more useful as more samples are analysed, mirroring how related samples surface over the course of a real investigation.

## 7.12 Network-Graph Intelligence

Rather than analysing domains, addresses, and URLs in isolation, the platform builds an interconnected threat graph linking applications, infrastructure, certificates, domains, addresses, and known campaigns. This reveals hidden relationships that isolated indicators miss, enabling rapid identification of shared infrastructure and coordinated attacks — a single malicious domain can expose an entire network of related threats.

## 7.13 Multi-Label Malware-Type Classifier

The classifier is an experimental component that takes the same feature set and independently predicts the likelihood of the APK belonging to one or more threat categories. It is multi-label by design, because a single application can simultaneously exhibit characteristics of several categories. The categories are:

**Banking Trojan · SMS Stealer · Spyware · Obfuscated Loader · Benign**

An independent classifier is trained for each category, producing a probability and a yes/no prediction. During training, an interpretability technique is used to identify which features drive each classification, so that the model's decisions are explainable rather than opaque — an important property in a security context, where “the model said so” is not sufficient. The classifier is treated strictly as a supplementary cross-check and does not influence the fusion verdict.

## 7.14 Report Generation

The final stage consolidates everything. It does not compute a new signal; instead it synthesises the outputs of every component — the two scores and the fusion verdict, the reverse-engineering findings, the social-engineering and impersonation results, the DNA family match, the campaign links, and the classifier output — into a single structured incident report written by the local language model.

The report transforms raw technical indicators into clear, actionable intelligence: a risk assessment and recommended action, an explanation of the evidence and the capability gap, the relationship to known threats and campaigns, a behavioural summary, and prioritised next steps. Two constraints govern its generation: the deterministic Feature-Store Score and the fusion verdict are authoritative and the report leads with them, while the Code-Behaviour score is presented as advisory; and the model is instructed to cite only evidence actually present in the analysis, never to invent capabilities, family names, or figures. The report enables security teams and decision-makers to rapidly understand not only what an application does, but why it is risky, how it relates to known threats, and what action should be taken next.

## 8. Leveraging Generative AI

---

Generative AI is applied deliberately and in bounded roles, never in a way that compromises the auditability of the deterministic verdict. A single locally hosted language model serves three distinct purposes across the pipeline.

### 8.1 Reverse engineering

The decompiled source produced by JADX is interpreted by the local model, enabling semantic understanding of application behaviour beyond what traditional static analysis can determine. The model identifies malicious workflows, suspicious logic, and attack techniques, and confirms behaviours that feed the Code-Behaviour Score and the MITRE mapping. Its output is constrained to the shared behaviour-tag vocabulary, so it slots directly into the deterministic scoring system rather than producing free-form text.

### 8.2 Social-engineering analysis

The same model analyses the application's interface text to recognise manipulation patterns — impersonation, urgency, counterfeit credential-entry screens — with a prompt tuned for the Indian banking context. Its findings are validated against the application's actual text so that nothing is fabricated.

### 8.3 Report generation

Findings from static analysis, dynamic analysis, the machine-learning classifier, the reverse-engineering stage, and the threat-intelligence components are consolidated by the model into a unified security report. It transforms technical indicators into structured, human-readable insight — risk summaries, attack explanations, family attribution, and recommended mitigations — for analysts and decision-makers.

### 8.4 Why local inference

Hosting the model locally keeps code analysis and reasoning entirely on-premises. This ensures privacy for potentially sensitive samples, removes any dependency on an external cloud AI service, and gives the bank full control over the model and its behaviour.

#### **Design discipline: AI never overrides the deterministic core**

The Generative-AI outputs are advisory and interpretive. The deterministic Feature-Store Score and the fusion verdict remain authoritative and fully reproducible. This separation is what allows the platform to use the strengths of Generative AI — reading code, recognising language patterns, writing clear reports — without sacrificing the auditability a financial-fraud decision demands.



## 9. Analyst Console

---

The platform is operated through a dedicated web console that turns the pipeline's output into a navigable investigation surface. The console is organised around three views and a set of evidence panels, with the analysis result presented as a coherent whole rather than a raw data dump.

### 9.1 Views

- **Dashboard** — The entry point and overview of the platform.
- **Analyse** — Where an APK is submitted. The analyst watches the pipeline advance in real time as each stage reports progress, then is taken to the result when the analysis completes.
- **Result** — The full analysis for a single APK, presenting the verdict, the two scores, and every component's evidence panel. Each panel can be opened on its own for a focused view.
- **History** — A list of every previously analysed APK, newest first, so any past analysis can be reopened.

### 9.2 Evidence panels

The result view composes the analysis from focused panels, each surfacing one component's findings: the two scores and the fused verdict; the MITRE ATT&CK coverage; the malware-type classification; the DNA family match; the campaign links; the social-engineering findings; the impersonation result; the confirmed behaviour catalogue; the capability gap; the reverse-engineering findings; the feature-store evidence; the network graph; and the consolidated incident report. Presenting each signal in its own panel keeps the evidence transparent and lets an analyst drill from the headline verdict down to the specific signals behind it.

### 9.3 Real-time progress

Because a full analysis takes minutes, the console subscribes to a live progress stream and shows the pipeline advancing stage by stage — static analysis, dynamic analysis, scoring, reverse engineering, and so on. The analyst sees exactly where the analysis is rather than waiting on an opaque request, and the final result is loaded automatically on completion.

## 10. Scalability

---

The architecture is designed so that the move from a single-machine proof-of-concept to a production-scale deployment is a matter of substituting infrastructure components, not redesigning the analysis logic. The key scaling steps are:

- **Parallel processing** — Move job orchestration to a distributed queue so that many analyses run concurrently across multiple workers rather than sequentially on one.
- **Scalable storage** — Move the feature and fingerprint stores to a database built for concurrent writes and proper indexing as the corpus grows.

- **Hosted model serving** — Replace local single-instance inference with a dedicated, horizontally scalable model-serving endpoint to handle concurrent analysis requests without queuing.
- **Approximate similarity search** — Introduce a vector index for fingerprint comparison once the analysed corpus is large enough that exact comparison becomes the bottleneck.
- **Containerised analysis services** — Run the analysis tooling as containerised services so that capacity can be scaled out horizontally once the processing queue is decoupled.

## 11. Future Scope

---

Several enhancements extend the platform's detection power, coverage, and operational maturity.

### 11.1 Stronger detection

- Train the malware-type classifier on real, labelled field data, using the platform's own feature extraction as the labelling infrastructure so that training and serving share an identical feature format.
- Incorporate dynamic runtime behaviour into the structural fingerprint, broadening the signal beyond static structure.
- Continuously expand the whitelist of genuine banking applications as new apps and certificate rotations occur.

### 11.2 Broader coverage

- Extend impersonation detection beyond banking apps to UPI apps, wallets, and government-facing services such as UMANG and DigiLocker.
- Add support for iOS application packages, since the feature-extraction and scoring architecture is platform-agnostic in principle.

### 11.3 Operational maturity

- Introduce an analyst feedback loop so that investigators can confirm or override verdicts, feeding corrections back as training signal.
- Automatically re-score previously analysed samples whenever a new malware family is added to the reference database.
- Integrate with FIU-IND and bank security-operations tooling for direct alert ingestion and standardised intelligence sharing.

## 12. Prototype

A working proof-of-concept has been built to demonstrate that the architecture functions end to end. As an idea-phase prototype, it carries deliberate limitations that the production design addresses, and they are stated here transparently.

**Git Hub Link:** <https://github.com/ShravniThakur/ThreatIntelligenceSystem>

**Demo Video of our Prototype:** <https://youtu.be/91jNUvVTX5Y?si=NsvkIM4dvp4vp0Ga>

**PPT:** <https://drive.google.com/file/d/1Qloz1V7KB3ZtauHAYCi3cm7kBDZ74Sdd/view?usp=sharing>

**BANK OF INDIA THREAT INTELLIGENCE**

Dashboard | Analyze APK | History

### Analysis Report

Analyze Another APK

Identity verified - no impersonation No impersonation of any known bank app detected.

**40 / 100** **upload\_0234c3438db94d598e10ae59461cc0ff.apk**  
SHA256: d9a33c8f6b447bf2c69f5f9c57f08c926ceb1d6c82437d9db9bf7ef9f4e871b  
Medium low grade monitor

**40 / 100** **FEATURE-STORE SCORE**  
Deterministic - what the app declares  
6 declared capabilities - same input → same score

**Medium** **CODE-BEHAVIOUR SCORE** (AI-derived - indicative)  
Reverse-engineered - what the code does  
band shown (raw 34) - 2 behaviours confirmed in code

**SCORE ANALYSIS** - quadrant: low grade - recommended action: monitor  
Low-grade signals only: Feature-Store 40/100 (Medium), Code-Behaviour Medium band. No high-severity capability declared or confirmed in code; code-only behaviours noted: c2\_sanctioned\_infra, hidden\_c2. Monitor.

Analysis Components | Threat Report

---

**BANK OF INDIA THREAT INTELLIGENCE**

Dashboard | Analyze APK | History

### Analysis Report

Analysis Components | Threat Report

MITRE ATT&CK Coverage 1 tactics

ML Threat Classification 1 predicted labels

Malware DNA — structural fingerprint closest: bankbot

Fraud Campaign — linked APKs Not detected

Social Engineering no fraud UI detected

Behaviour Catalog 2 confirmed in code

Capability Gap 2 undeclared behaviours

Network & C2 Surface C2 / endpoint graph

Reverse-Engineering Findings 8 methods analyzed

Copy text | Print / Save as PDF

#### Executive Summary

This analysis of the `con.example.dynamictestapp` APK reveals a medium-risk application exhibiting several concerning behaviors despite lacking explicit indicators of malicious intent. The Feature Store Score of 40/100 reflects this cautious assessment, primarily driven by confirmed C2 web activity and obfuscation alongside other potentially suspicious code elements. While no high-severity vulnerabilities were definitively identified through code analysis, the presence of hidden C2 infrastructure and a structurally similar “bankbot” family warrants close monitoring and further investigation to determine if this app is a benign test application or a nascent threat actor’s tool. The recommended action is to monitor for any unusual network activity or behavior changes.

#### 1. Verdict & Recommended Action

**Verdict:** Low-Grade Risk - Monitor

**Recommended Action:** Implement continuous monitoring of the application’s network traffic, file system access, and runtime behavior. Specifically, prioritize inspection of network connections, focusing on web protocols (T1437.001), and scrutinize any attempts to access or modify local files, particularly those related to debugging or configuration. Given the structural similarity to the “bankbot” family, a deeper analysis of its functionality is recommended.

**Rationale:** The fusion verdict reflects the limited evidence available. The Feature Store Score of 40/100 indicates a moderate level of concern based on declared capabilities and code-confirmed behaviors. The Code-Behaviour band of Medium suggests that while the app exhibits some

**Known limitations of the current proof-of-concept:**

- Storage uses a local lightweight database, suitable for demonstration but not for production-scale workloads.
- The malware-type classifier is trained on synthetic data; its predictions are illustrative and are deliberately excluded from the final verdict.
- Generative-AI inference runs a small local model, giving lower quality and throughput than a production-grade model would.
- The whitelist of genuine banking apps covers a limited set.
- The service runs as a single worker, which prevents horizontal scaling in its current form.
- Throughput is constrained by consumer-grade hardware and the largely sequential execution of the pipeline.

## 13. Conclusion

The APK Threat Intelligence System answers the problem statement with an automated, explainable, Generative-AI-assisted platform for analysing fraudulent banking APKs. Its defining strength is architectural discipline: a deterministic, fully auditable risk score that a bank can defend, paired with Generative-AI reasoning that reads code, recognises manipulation, and writes clear reports — with the two kept rigorously separate. By comparing what an application declares against what its code actually does, the system catches the obfuscated and hidden-payload threats that single-lens approaches miss, while a family of complementary detectors adds breadth across impersonation, malware-family resemblance, coordinated campaigns, and social engineering.

The result is a platform that enables faster, more consistent identification of fraudulent applications, grounds its findings in an internationally recognised threat framework, and produces analyst-ready intelligence — supporting proactive cybersecurity and fraud prevention for the bank, and built so that the path from prototype to production scale is clear.

***Let's build something that lasts.***

***Thank You.***